#4 Initialize a single reducer, that will run `canopyClustering` on the set of all centroids returned by all mappers. This node also will be alive for the whole method's lifetime, but it won't hold any data.
#5 Repeat until convergence (it might be a good idea to set a max number of iterations, and also save the closest result found).
#6 Run MapReduce, using the mappers and reducers already created. Mappers will receive the current values for the thresholds (in addition, each mapper already hold one shard of the dataset points). The single reducer will read their input (the canopy centroids selected for each shard) from the mappers, and returned a refined list of centroids.
#7 If the number of centroids matches the desired result, break out of the cycle.
#8 Otherwise, check if the algorithm returned more centroids than needed.
#9/10/11 We need to get fewer centroids, so we can try raising the threshold so canopies will be larger and each hold more points in their inner perimeter. Since `T1` must be larger than `T2`, we need to increment that value as well, to be sure. The random value added needs to be some function of `T2`, to be sure that the delta is meaningful.
#12 If, instead, we need more centroids, we can try making the inner threshold smaller.
#13 Return the centroids, after adding some random noise to them (as described in chapter 12, for k-means we might want to select centroids close to the dataset points but not exactly the ones in the dataset).